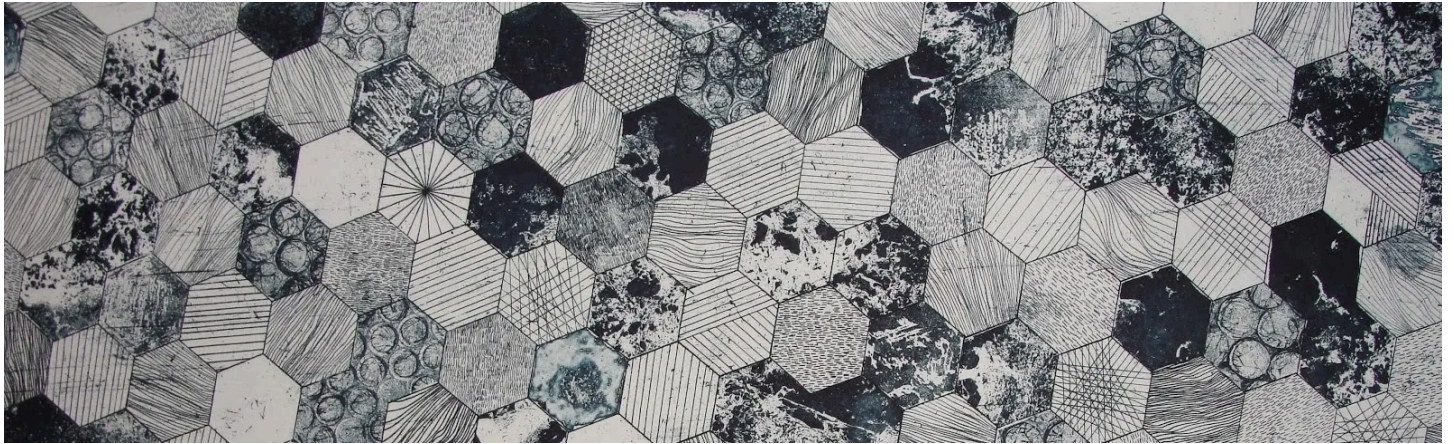




Intro to Modern Bayesian Learning and Probabilistic Programming

8 min read · Jan 16, 2019



Petuum, Inc.

Follow



Listen



Share

By Willie Neiswanger

What is Bayesian machine learning?

To answer this question, it is helpful to first take a look at what happens in typical machine learning procedures (even non-Bayesian ones). In nearly all cases, we carry out the following three steps:

1. Define a model: This is usually a family of functions or distributions specified by some unknown model parameters.
2. Pick a set of data.
3. Run a learning algorithm: This means using the data to choose a value for the unknown model parameters.

Take deep neural networks as an example: You start with a family of functions with unknown weight parameters, throw in data — usually a large quantity of (input,

output) pairs — and then run some gradient-based optimization algorithm (e.g., backpropagation) to get a final set of weights.

In Bayesian machine learning, we roughly follow these three steps, but with a few key modifications:

1. To define a model, we provide a “generative process” for the data, i.e., a sequence of steps describing how the data was created.
 - This generative process includes the unknown model parameters.
 - We incorporate our prior beliefs about these parameters, which take the form of distributions over values that the parameters might take.
2. Data are viewed as observations from the generative process.
3. After running the learning algorithm, we are left with an updated belief about the parameters — i.e., a new distribution over the parameters.

Why would you want to do this, and when is it useful?

The Bayesian strategy is particularly useful when:

- You have prior beliefs about unknown model parameters or explicit information about data generation — i.e., useful info you want to incorporate.
- You have few data or many unknown model parameters and it is hard to get an accurate result with your data alone (without the added structure or information).
- You want to capture the uncertainty about your result — how sure or unsure your model is — instead of only a single “best” result.

For example, Bayesian learning is used in a variety of industry settings where there are few data and uncertainty quantification is critical, including marketing, advertising, medical product development, pharmaceutical statistics, drug discovery and development, technical recruiting, and computer system A/B testing and tuning.

A concrete example: Has my milk gone bad?

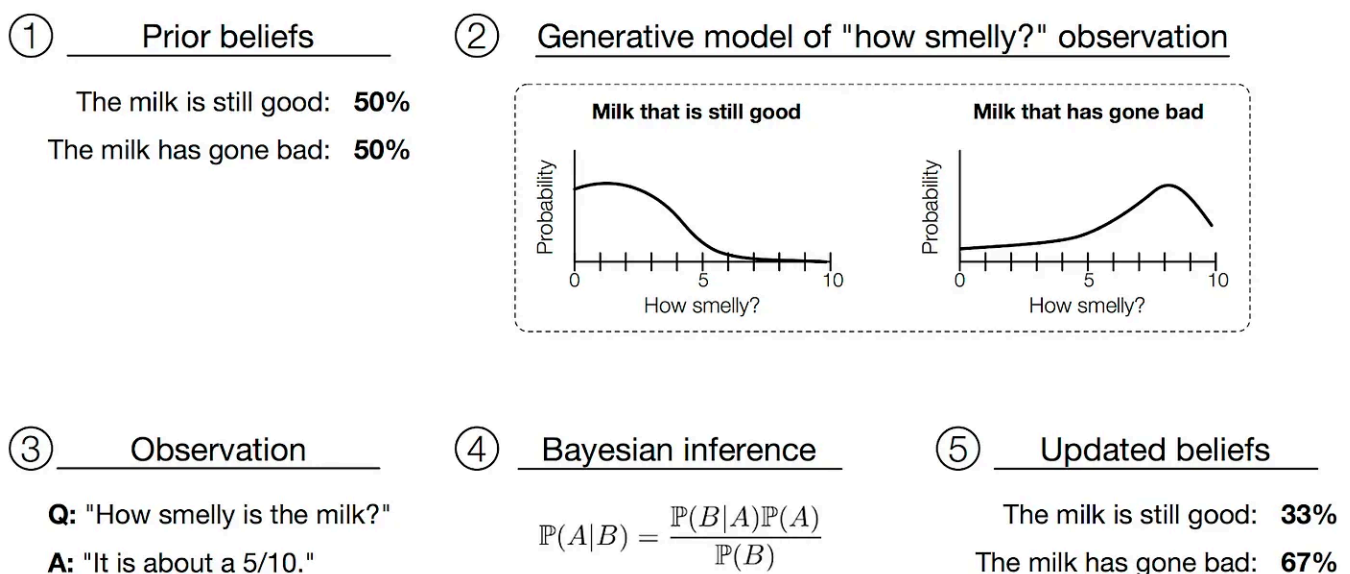
We can use Bayesian learning in daily life: Suppose I grab a carton of milk from the fridge, see that it is seven days past the expiration date, and want to know if the milk is still good or if it has gone bad. A quick internet search leads me to believe that

there is roughly a 50–50 chance that the milk is still good. This is my prior belief (Figure 1).

From past experience, I have some knowledge about how smelly milk gets when it has gone bad. Specifically, let's suppose I rate smelliness on a scale of 0–10 (0 being no smell and 10 being completely rancid) and have probability distributions over the smelliness of good milk and of bad milk (Figure 2).

Here's how Bayesian learning works: When I get some data, i.e., when I smell the milk (Figure 3), I can apply the machinery of Bayesian inference (Figure 4, math shown in the next section) to compute an updated belief about whether the milk is still good or has gone bad (Figure 5).

For example, if I observe that the milk is about a 5 out of 10 on the smelly scale, I can then use Bayesian learning to factor in my prior beliefs and the distributions over smelliness of good vs. bad milk to return an updated belief — that there is now a 33% chance that the milk is still good and a 67% chance that the milk has gone bad.



What is probabilistic programming?

In recent years, there has been a surge in the popularity and development of probabilistic programming languages (PPLs) or frameworks (such as [Stan](#), [PyMC](#), [Pyro](#), [Edward](#), [Infer.NET](#), [WebPPL](#), [Anglican](#), and [many more](#)). At a very high level, PPLs:

1. Provide a way for users to write down a Bayesian model, including the generative process, unknown model parameters, and prior beliefs about these

parameters.

2. Allow a user to specify a dataset of interest.
3. Automatically compute and return the result distribution over model parameters.

For example, let's again consider the expired milk situation above. Suppose I want a program that quickly computes an updated belief about whether the milk has gone bad whenever I smell the milk. A PPL would allow me to do the following:

1. Encode my prior beliefs about whether the milk is still good or has gone bad and probability distributions over the smelliness of good vs. bad milk.
2. Smell the milk and give this observation as an input to the program.
3. Do Bayesian learning automatically and return an updated belief about whether or not the milk has gone bad.

Computing the updated beliefs for this milk situation roughly involves the following:

④ Bayesian inference

Define:

$A1$ = "Milk is still good", $A2$ = "Milk has gone bad", B = "Smell is 5/10"

Then:

$P(A1 B)$ = Posterior("Milk is still good" "Smell is 5/10")	$P(A2 B)$ = Posterior("Milk has gone bad" "Smell is 5/10")
= Prob("Smell is 5/10" "Milk is still good")	= Prob("Smell is 5/10" "Milk has gone bad")
× Prior("Milk is still good")/Normalization	× Prior("Milk has gone bad")/Normalization
= $0.1538 \times 0.5 / 0.2307$	= $0.3076 \times 0.5 / 0.2307$
= 0.3333	= 0.6667

Where we've used:

$$\begin{aligned}\text{Normalization} &= P(B|A1) \times P(A1) + P(B|A2) \times P(A2) \\ &= \text{Prob}(\text{"Smell is 5/10"}|\text{"Milk is still good"}) \times \text{Prior}(\text{"Milk is still good"}) \\ &\quad + \text{Prob}(\text{"Smell is 5/10"}|\text{"Milk has gone bad"}) \times \text{Prior}(\text{"Milk has gone bad"}) \\ &= 0.1538 \times 0.5 + 0.3076 \times 0.5 = 0.0769 + 0.1538 = 0.2307\end{aligned}$$

In the past, each time you wrote down a new Bayesian model, you would need to mathematically derive an inference algorithm — i.e., the learning algorithm that computes the final distribution over beliefs given the data. This process required (often a great deal of) expert human work for each new model. Now, you simply write down the model in your PPL and it returns the result automatically, with minimal human work.

Get Petuum, Inc.'s stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

At least that's how it should work, but this lofty goal is elusive. There are still a variety of settings where fully automatic and accurate Bayesian inference via probabilistic programming is difficult to achieve.

Current challenges (and research problems) in Bayesian learning and probabilistic programming

There is a vibrant community of researchers studying the areas in which Bayesian inference and probabilistic programming meet challenges.

Big data and big models

Bayesian inference can be computationally expensive. This is especially true when you have big data (large datasets) or big models (many unknown parameters). There has been a great deal of research into strategies for mitigating this issue, including:

- Making the algorithms work more efficiently for big datasets. This involves developing new inference algorithms that can better scale to large datasets without getting too computationally expensive [1][2][3].
- Making use of multiple computers. This involves the development of parallel algorithms, which can allow multiple computers to work well together. These algorithms also allow for learning in “data-distributed settings” in which data are split up and processed in separate groups due to large size, distributed collection procedures, or privacy concerns [4][5][6].
- Moving the goalposts. This means computing an approximation to the correct answer, which can be done more efficiently. This includes research into various types of approximations including work on variational inference and amortized inference [7][8][9][10].

Models with new and complex components

To tackle an ever-growing array of tasks, and armed with an ever-growing amount of computational resources, there is a similarly growing interest in more sophisticated Bayesian models. For example, there are now ways to use new types of model components when defining a Bayesian model or PPL, including:

- Deep neural networks [11][12][13].
- Simulators — e.g., of scientific processes or equipment [14][15].
- Visual graphics engines [16][17].
- Any arbitrary computer program [18][19][20].

However, the standard general-purpose algorithms built into most PPLs aren't always accurate or efficient for these sophisticated models. Sometimes, the built-in algorithms cannot be applied and novel techniques must be developed. PPLs are continuously being extended to incorporate new methods for new and complex models.

Automatic verification and quality assurance

It is hard to verify that a PPL has produced a correct result (sometimes, it is hard to even define what suffices as a correct, or correct-enough, result), or, given a result, it is hard to assess its quality. In order for PPLs to be robust components of a computational pipeline, it is important that they consistently produce accurate results and that we can make guarantees about (or at least give an estimate of) the quality of results [21][22].

Usability for the masses

Improving other aspects of the general usability of PPLs can allow for much broader use and adoption of these systems.

- Writing models often requires experienced statisticians or computer scientists — i.e., models are bespoke and it takes effort to learn how to design them. One area of research is the development of more intuitive, easier-to-use frameworks for creating models [23][24][25].
- Instead of monolithic, stand-alone systems, it is beneficial for PPLs to be designed as standardized components that can fit into the broader machine learning ecosystem and be used with other tools and frameworks — e.g., deep learning and other ML techniques and software packages. One research goal is to make PPLs more modular and standardized [26][27].

At Petuum, we are pursuing a number of these research challenges, including developing Bayesian inference algorithms for PPLs that scale up to big data/models; allowing for deployment on large numbers of computers via parallel algorithms; increasing general usability and robustness guarantees; and achieving better inference performance automatically in a large range of settings.

Further Reading

The following are some recommended papers cited throughout this blog post, broken down into categories:

Scalable Bayesian inference algorithms:

[1] [Bayesian Learning via Stochastic Gradient Langevin Dynamics](#)

[2] [Stochastic Gradient Hamiltonian Monte Carlo](#)

[3] [Big Learning with Bayesian Methods](#)

Parallel and distributed Bayesian inference algorithms:

[4] [Asymptotically Exact, Embarrassingly Parallel MCMC](#)

[5] [Parallelizing MCMC via Weierstrass Sampler](#)

[6] [Scalable and Robust Bayesian Inference via the Median Posterior](#)

Variational approximations and amortized inference:

[7] [Variational Inference: A Review for Statisticians](#)

[8] [Stochastic Variational Inference](#)

[9] [Stochastic Backpropagation and Approximate Inference in Deep Generative Models](#)

[10] [Auto-Encoding Variational Bayes](#)

Deep Bayesian learning:

[11] [Deep Probabilistic Programming](#)

[12] [Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning](#)